

Tesina Finale di  
**Programmazione di Interfacce Grafiche e Dispositivi Mobili**  
Corso di Laurea in Ingegneria Informatica ed Elettronica – A.A. 2025-2026  
DIPARTIMENTO DI INGEGNERIA

**JPvZ**  
applicazione desktop JFC/SWING



## Sommario

|                                                        |    |
|--------------------------------------------------------|----|
| 1. Descrizione del Problema .....                      | 2  |
| 1.1 Il Videogioco Plants vs Zombies 2 .....            | 2  |
| 1.2 L'applicazione JPvZ.....                           | 2  |
| 2. Specifica dei Requisiti .....                       | 3  |
| 3. Progetto .....                                      | 4  |
| 3.1 Architettura software .....                        | 4  |
| 3.2 Model .....                                        | 5  |
| 3.3 View .....                                         | 7  |
| 3.4 Controller .....                                   | 10 |
| 3.5 Problemi riscontrati .....                         | 12 |
| 4. Possibilità di estensione e personalizzazione ..... | 12 |
| 5. Bibliografia.....                                   | 13 |

# 1. Descrizione del Problema

L'obiettivo di questo lavoro è lo sviluppo di un'applicazione desktop, denominata *JPvZ*, che realizza una versione estremamente semplificata dell'omonimo videogioco, mantenendo pressoché simile il funzionamento delle caratteristiche principali. L'applicazione verrà implementata utilizzando la tecnologia JFC/Swing.

Di seguito sarà data una breve descrizione del videogioco originale, dopodiché si fornirà una descrizione della versione semplificata che si intende realizzare.

## 1.1 Il Videogioco Plants vs Zombies 2

Plants vs. Zombies 2 è un videogioco strategico di genere tower defense pubblicato da Electronic Arts e sviluppato originariamente da PopCap Games. Il giocatore ha l'obiettivo di difendere la propria casa dall'avanzata di orde di zombi, e in questo secondo titolo la progressione si sviluppa attraverso veri e propri viaggi nel tempo, portando le battaglie in epoche storiche differenti come l'Antico Egitto, il Far West o i Mari dei Pirati. Il campo di battaglia è rappresentato da una griglia suddivisa orizzontalmente in righe e colonne.

Per arginare l'avanzata dei nemici (che si muovono da destra verso sinistra in linea retta), il giocatore deve schierare un arsenale composto da piante e funghi, ognuno caratterizzato da abilità difensive, di supporto o offensive uniche, oltre all'estrema linea di difesa costituita da tagliaerba, a singolo utilizzo, i quali se attivati da uno zombie che arriva nei suoi pressi, elimina tutti i nemici presenti in quella linea. L'intera meccanica di gioco ruota attorno a un sistema economico dinamico: per posizionare le truppe sulla scacchiera è necessario accumulare una risorsa, il "sole", che viene generata a intervalli regolari da alcune piante specifiche, come i Girasoli, grazie alla quale si possono comprare le piante per la difesa.

Il livello di sfida è progressivo e impone un approccio molto tattico. Ogni tipologia di zombi possiede infatti comportamenti peculiari, resistenze e debolezze specifiche (ad esempio nemici in grado di saltare le difese o protetti da armature). Questo costringe il giocatore a pianificare in anticipo, selezionando con cura il "mazzo" di piante da portare con sé all'inizio di ogni schema. La regola per la sconfitta è spietata: se anche un solo nemico riesce a superare le difese e a oltrepassare il margine sinistro dello schermo, incluso il tagliaerba, la partita è perduta.

## 1.2 L'applicazione JPvZ

L'obiettivo di questo progetto è lo sviluppo di un'applicazione desktop che simuli il comportamento del gioco Plants vs Zombies 2 citato sopra.

Come prima schermata questa applicazione presenta un menu principale grazie al quale l'utente può decidere se consultare lo score, per visualizzare i punteggi migliori ottenuti, le impostazioni, per modificare difficoltà o rimuovere i suoni, o la modalità play, per avviare la sessione di gioco vera e propria.

Una volta avviata la partita tramite la modalità play, l'utente si ritroverà un campo di battaglia rappresentato visivamente da una mappa a griglia. Nella parte superiore dello schermo è posizionata una barra che fornisce all'utente gli strumenti necessari alla difesa della propria abitazione: un inventario di piante schierabili, uno strumento dedicato (la pala) per rimuovere (spiantare) eventuali unità già posizionate sulla mappa, e un indicatore economico che traccia il saldo dei "soli" (la valuta del gioco) indispensabile per poter posizionare le varie piante.

Durante lo svolgimento del livello verranno generati nemici (zombie) dinamicamente attraverso orde (gruppi di zombie), modellate in base alla difficoltà scelta dall'utente.

Lo scopo del gioco è quello di gestire dinamicamente le risorse e le difese grazie alle quali poter sconfiggere i nemici (zombie) per poi poter vincere la partita, la quale si concluderà con una vittoria solo se tutti gli zombie generati dinamicamente saranno stati sconfitti con successo.

## 2. Specifica dei Requisiti

I requisiti richiesti all'applicazione nel suo complesso sono:

1. Il programma deve prevedere una schermata iniziale di benvenuto (Menù) che permetta all'utente di navigare tra diverse funzionalità indipendenti: avviare una nuova partita ("Play"), accedere alla pagina delle opzioni ("Impostazioni"), e consultare i punteggi ottenuti ("Score").
2. L'interazione con l'utente si deve svolgere il più possibile per via grafica, privilegiando l'utilizzo del solo mouse per gestire tutte le funzionalità di gioco.
3. L'interfaccia deve essere pulita e comprensibile, mettendo immediatamente a disposizione dell'utente tutti i controlli necessari (come il saldo dei "soli" e l'elenco delle piante schierabili).
4. La mappa di gioco (griglia) su cui si svolge la battaglia deve essere l'elemento dominante dell'interfaccia grafica. La rappresentazione delle entità (piante e zombi) deve essere chiara ed immediata.
5. Tutti i movimenti degli elementi sulla scacchiera (il cammino degli zombi, il volo dei proiettili, la caduta dei soli) devono essere presentati all'utente tramite delle animazioni, le quali devono essere fluide.
6. Il sistema deve supportare la riproduzione di effetti sonori. Essendo un gioco dinamico, deve essere garantita la corretta sovrapposizione dell'audio quando avvengono più eventi sonori in contemporanea.
7. Tramite un opportuno file di configurazione (e l'apposita schermata Impostazioni) deve essere possibile modificare i parametri vitali del gioco per settare il livello di difficoltà.
8. L'utente interagisce con la griglia facendo click con il mouse sulla casella in cui desidera posizionare una nuova pianta o spiantarne una esistente
9. L'utente deve poter raccogliere le risorse economiche ("soli") necessarie per giocare cliccandoci sopra con il mouse.
10. L'applicazione deve evidenziare in ogni momento all'utente quali sono le caselle della scacchiera in cui è consentito o meno posizionare una specifica unità (evitando sovrapposizioni).

11. L'applicazione deve avvertire con una notifica la fine della partita, che corrisponde alla vittoria (se tutte le ondate vengono respinte) o al Game Over (se un nemico raggiunge il margine sinistro dello schermo)
12. Deve essere possibile mettere in pausa la partita.
13. Il sistema deve generare e gestire autonomamente l'accumulo spontaneo della risorsa "sole" ad intervalli regolari.
14. La parte dell'applicazione che si occupa dell'intelligenza dei nemici deve generare autonomamente le orde di zombi ad intervalli prefissati, facendoli avanzare in linea retta dal margine destro verso quello sinistro in modo continuo
15. Il sistema deve calcolare in modo invisibile all'utente le collisioni tra le entità: se un proiettile colpisce uno zombi, o se uno zombi entra in contatto con una pianta, il sistema deve scalarne i punti vita fino alla distruzione dell'entità stessa.

## 3. Progetto

Ora si passa alla trattazione della struttura dell'applicazione realizzata, mostrando prima l'architettura del software utilizzata e, in seguito, alla descrizione più approfondita dei blocchi funzionali che la compongono.

### 3.1 Architettura software

Per la realizzazione dell'applicazione JpVz si è deciso di basarsi sull'utilizzo del pattern di programmazione MVC (Model View Controller). Quindi il software si divide in tre blocchi:

- Il Model si occupa di tutta la logica e i dati dell'applicazione, in questo caso della gestione delle collisioni, di aggiornamenti di posizione e stato dei vari elementi, della possibilità o non di eseguire delle azioni e notifica tutti i cambiamenti al View tramite il Controller.
- Il View si occupa di rappresentare visivamente i dati e gli aggiornamenti di essi ricevuti dal Model, raccogliendone gli input dall'utente.
- Il Controller funge da intermediario tra i due moduli, intercetta gli input dalla View, richiede le modifiche al Model e in seguito, richiede di aggiornare la grafica al View.

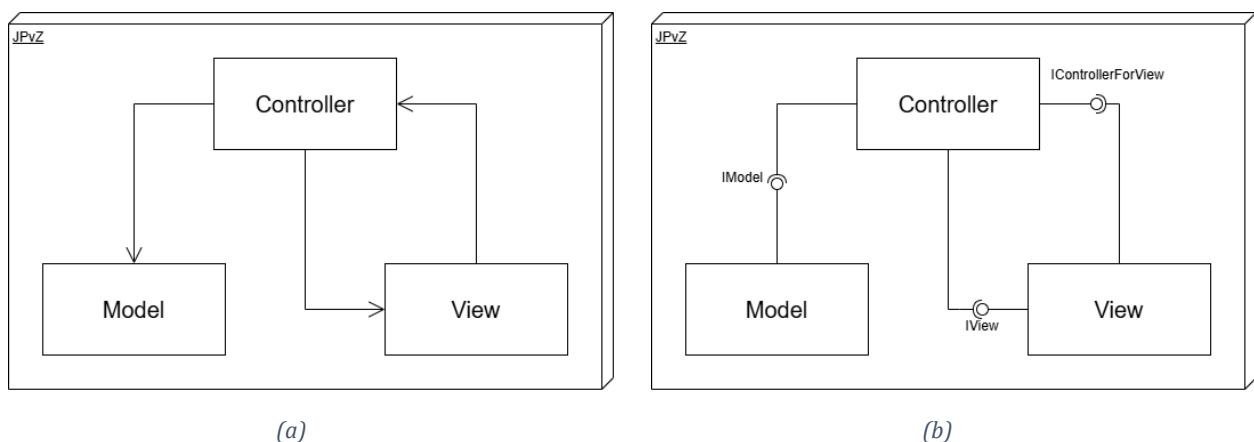


Figura 3.1: Architettura Model View Controller (MVC) di JpVz ove sono evidenziate: (a) le direzioni dei flussi informativi, ovvero come i vari moduli comunicano tra di loro, inoltre, questi moduli interagiscono tra di loro usando delle interfacce (b) limitando ulteriormente l'accoppiamento tra classi di moduli distinti.

Oltre alle classi contenute nei moduli descritti e rappresentati nel precedente schema l'applicazione contiene un modulo di Utilities. Questo modulo contiene: una classe di supporto per l'interfaccia utente che fornisce delle componenti già preparate e più classi per caricamento di immagini, suoni, impostazioni (dal file di configurazione) e punteggi.

Di seguito vengono descritti nel dettaglio Model, View e Controller e le rispettive classi.

## 3.2 Model

Le classi appartenenti al blocco Model si trovano nel package jpvz.model e la loro struttura è rappresentata nel diagramma UML di seguito, (figura 3.2) che esplora gli attributi e metodi principali di ogni classe. Sono presenti anche delle sottoclassi delle classi Plant e Zombie situate in due sotto-package del model, rispettivamente jpvz.model.plants e jpvz.model.zombies e contenenti 12 tipi di piante e 9 tipi di zombie.

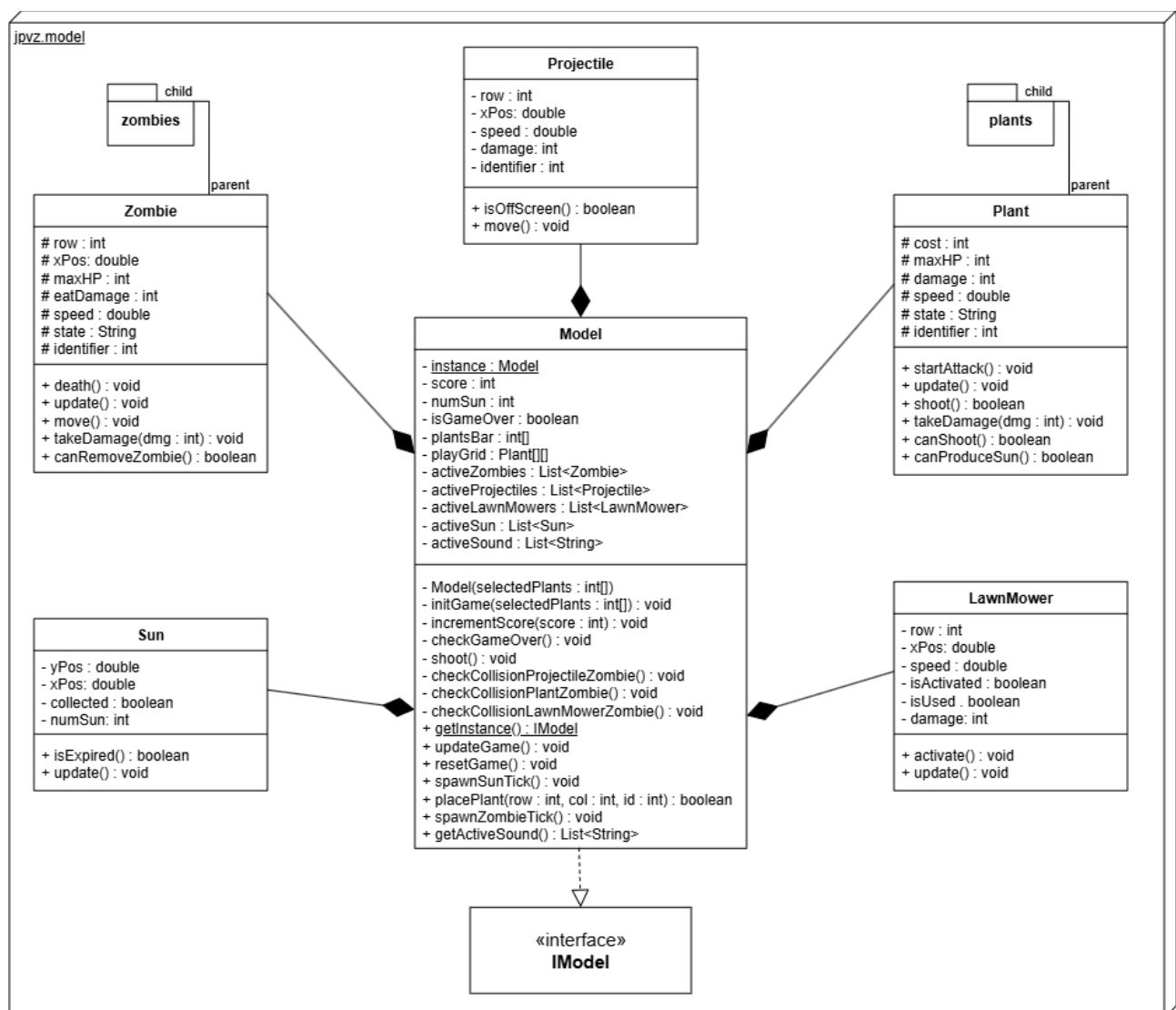


Figura 3.2: Diagramma UML delle classi contenute nel package model (jpvz.model)

In particolare:

- **Plant**: superclasse che modella le piante che è possibile posizionare sulla griglia di gioco. Definisce gli attributi comuni a tutte le sottoclassi, metodi che permettono di gestire l'aggiornamento di stato e del ciclo di vita (chiamati ad ogni tick) e metodi per definire le capacità specifiche della pianta come la possibilità di sparare o produrre soli.
- **Zombie**: superclasse che rappresenta tutti gli zombie che sono presenti nel gioco. Definisce attributi comuni a tutte le sottoclassi oltre a metodi che permettono di gestire l'aggiornamento di stato e del ciclo di vita (chiamati ad ogni tick) e altri che gestiscono le meccaniche di movimento e di ricezione e assorbimento dei danni.
- **Projectile**: classe che rappresenta i proiettili che vengono sparati nel gioco. Ha un metodo per verificare se il proiettile è fuori dalla visuale di gioco e un altro che ne gestisce il movimento, con comportamenti diversi in base alla pianta dalla quale viene sparato.
- **Sun**: classe che rappresenta i soli che nascono spontaneamente o dalle piante nella griglia di gioco. Contiene metodi che ne permettono la gestione dell'aggiornamento di stato e della posizione.
- **LawnMower**: classe che rappresenta i tagliaerba all'inizio della griglia di gioco. Contiene metodi che ne permettono la gestione dell'aggiornamento di stato e della posizione.

**Model**, essendo la classe centrale del model, merita una trattazione più dettagliata. Questa classe, implementata tramite pattern singleton, coordina e mantiene lo stato e la logica della partita. Implementa l'interfaccia **IModel**, la quale dichiara quali metodi della classe Model possono essere chiamati in altri package per limitare l'accoppiamento tra le varie classi. Contiene attributi grazie ai quali si gestisce lo stato globale della partita e altri per gestire tutte le entità attive nella griglia di gioco. Implementa metodi per la manipolazione degli attributi precedentemente citati e altri per l'aggiornamento, inizializzazione e reset della partita. In particolare, tra i principali troviamo:

- `updateGame()`: metodo che funge da fulcro operativo del model poiché richiama altri metodi privati della stessa classe Model che gestiscono le collisioni, aggiornamenti di tutte le entità attive e di condizioni come `checkGameOver()`.
- `initGame()`: metodo che si occupa di inizializzare la griglia di gioco, le piante selezionate, tutte le liste delle entità e le condizioni necessarie per lo stato del gioco.
- `resetGame()`: metodo che si occupa di riportare allo stato iniziale tutti gli attributi per poter iniziare una nuova partita.
- `placePlant()`: metodo che viene chiamato, attraverso comunicazione tra View e Controller, quando il giocatore vuole posizionare delle piante.
- `spawnZombieTick()`: metodo che gestisce l'avanzamento di orde e zombie, variandone il numero in base alla difficoltà selezionata.
- `getActiveSound()`: in base allo stato delle entità o ad azioni scaturite dall'utente, vengono aggiunte delle parole chiave in una lista di stringhe per identificare gli specifici eventi. Questo metodo restituisce la lista di stringhe per poi delegare la riproduzione di suoni al controller.

Va notato che per la gestione delle entità a schermo, la classe si affida a due strutture dati diverse: una matrice bidimensionale di tipo Plant per gestire e rappresentare la griglia di gioco, dato che deve avere dimensioni fisse, mentre per la gestione delle altre entità, che non hanno una dimensione prestabilita, ci si affida a una serie di liste dinamiche, le quali vengono

aggiornate continuamente dal metodo `updateGame`, precedentemente descritto, invocato regolarmente da un timer nel View.

### 3.3 View

Le classi appartenenti al blocco View di JpVZ si trovano nel package `jpVz.view`. La loro struttura è rappresentata nel diagramma UML di seguito (figura 3.3).

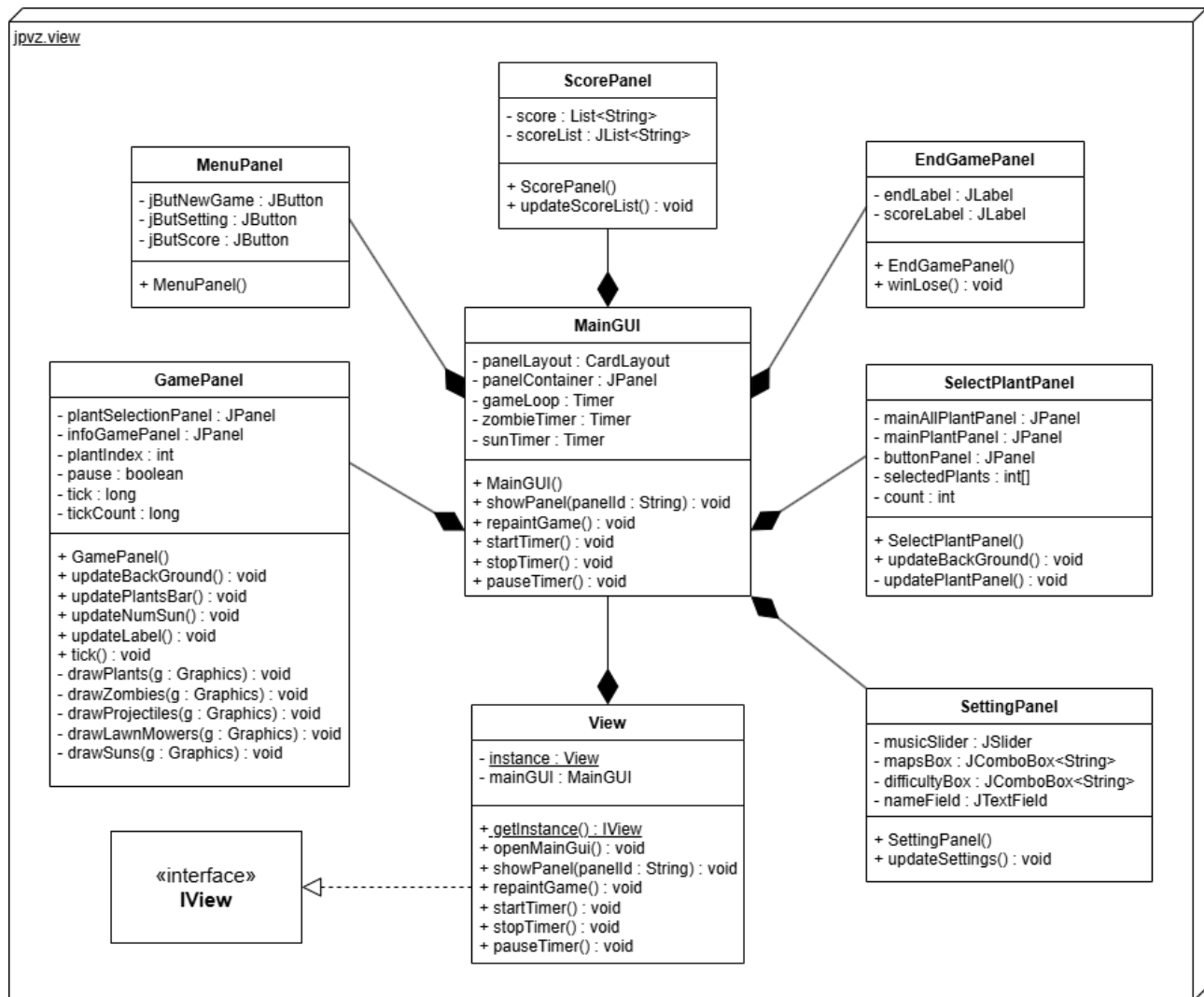


Figura 3.3: Diagramma UML delle classi contenute nel package view (`jpVz.view`)

La classe `MainGUI` estende `JFrame` ed è l'unica finestra grafica con la quale si interfaccia l'utente. Al suo interno si trovano sei pannelli: `MenuPanel`, `ScorePanel`, `SettingPanel`, `SelectPlantPanel`, `GamePanel` e `EndGamePanel`, ciascuno dei quali estende `JPanel` e rappresenta una schermata specifica dell'applicazione, entrando nello specifico:

- **MenuPanel**: è la schermata iniziale del gioco, contiene un insieme di bottoni per consultare i punteggi, per cambiare alcune impostazioni di gioco e per passare alla schermata di selezione delle piante prima di iniziare la partita. Inoltre, contiene un `JTextField` dove l'utente può subito inserire il suo nome.

- **ScorePanel**: è la schermata con lo storico dei punteggi dove si visualizza un pannello scorrevole contenente una JList di stringhe formate da nome del giocatore e punteggio effettuato, ordinate in ordine decrescente. Inoltre, è presente un bottone per tornare alla schermata iniziale di menù.
- **SettingPanel**: è la schermata dove si possono modificare alcune impostazioni di gioco come: volume della musica, livello di difficoltà, nome dell'utente e mappa di gioco che vengono salvate appena l'utente seleziona il bottone in fondo al pannello. Inoltre, si possono modificare altri valori dal file di configurazione (config.txt) e non da questa schermata perché ritenuti parametri accessori.
- **SelectPlantPanel**: schermata nella quale l'utente può selezionare, tra un insieme di 12 piante, le 6 che vuole portare con sé nella partita. Sono presenti inoltre, due bottoni, uno per tornare indietro nella schermata iniziale di menu e, uno, per proseguire nella schermata di gioco.

**GamePanel**, svolgendo il ruolo fondamentale di rappresentare la schermata di gioco, merita una descrizione più dettagliata. La schermata si presenta visivamente come un insieme di tre pannelli:

- al centro, la mappa di gioco dove si trovano tutte le entità descritte in precedenza.
- in alto un pannello che contiene le piante disponibili, il saldo dei soli e la pala per rimuovere le piante indesiderate.
- In basso alcune informazioni riguardanti lo stato della partita e il nome dell'utente, oltre a due bottoni che permettono di mettere in pausa la partita e di tornare al menu.

Il rendering della schermata viene scandito ad intervalli regolari dal timer di gioco, che chiama il repaint dell'intero GamePanel. Il pannello contenente la mappa di gioco effettua l'override del metodo paintComponent(Graphics g), il quale richiama in ordine i metodi:

- drawPlants(): si occupa di disegnare le piante ognuna nella propria casella della griglia di gioco, con il rispettivo stato. Infatti, in base allo stato della pianta, viene anche riprodotta una sequenza di frame corrispondenti all'azione che sta compiendo (come l'attacco, l'esplosione o la crescita) grazie al conteggio dei tick che variano in base alle specifiche azioni.
- drawZombies(): si occupa di disegnare gli zombie durante il loro avanzamento dall'estremo destro della mappa verso quello sinistro. In base allo stato di ogni zombie vengono selezionate le sequenze di frame appropriate alle diverse azioni che vengono sincronizzate grazie al conteggio dei tick.
- drawProjectiles(): si occupa del disegno dei proiettili che avanzano dalla rispettiva pianta verso lo zombie più vicino nella stessa riga o verso la fine della mappa.
- drawLawnMowers(): si occupa del disegno dei tagliaerba che si muovono dall'estremo sinistro della mappa fino alla fine, non appena vengono attivati.
- drawSuns(): si occupa del disegno dei soli attivi, che compaiono dal cielo in una colonna casuale se generati dal timer di gioco, oppure sopra la pianta se nati dalla stessa. Questi si muovono verso il basso e verso destra fino a che non vengono raccolti dal giocatore o non svaniscono.

Inoltre, questo pannello implementa anche un ascoltatore per il mouse (MouseAdapter) che ha tre funzioni principali:

- raccolta di soli: verifica se le coordinate del click e della hitbox del sole corrispondono per raccoglierlo
- interazione con la griglia: traduce le coordinate del mouse in coordinate discrete di riga e colonna corrispondenti alla griglia di gioco, per permettere al giocatore di posizionare una pianta selezionata o rimuoverne una tramite la pala.
- controllo dinamico della possibilità o meno di posizionare piante nella griglia di gioco: se l'utente seleziona una pianta e passa con il cursore del mouse su una casella già occupata, verrà visualizzato sulla stessa un colore rosso trasparente per evidenziare l'impossibilità del completamento dell'azione.

Infine, questa classe ha metodi di aggiornamento di statistiche dinamiche che vengono chiamati ad ogni tick.

- **EndGamePanel**: è la schermata che notifica la vittoria o la sconfitta del giocatore mostrando il punteggio e un bottone per tornare al menu. Il pannello viene mostrato in sovrapposizione alla schermata di gioco grazie ad un GlassPane, il quale, essendo trasparente, permette di lasciare visibile lo stato di gioco sottostante.

Tornando alla descrizione di MainGUI, i pannelli precedentemente descritti si alternano grazie all'uso di CardLayout panelLayout che viene aggiunto al JPanel panelContainer, al quale vengono aggiunti tutti i pannelli sopra citati solo dopo essere stati istanziati nel costruttore. Questi pannelli si alternano grazie al metodo showPanel(), che riceve in ingresso una stringa che identifica il pannello da mostrare e, in base a questa, aggiorna anche alcuni attributi del rispettivo pannello. La classe inoltre è dotata di attributi come gameLoop (scandisce il timer di gioco), zombieTimer (scandisce la nascita di zombie e orde, modificandone anche la frequenza) e sunTimer (scandisce la nascita dei soli dal cielo), i quali vengono sempre chiamati insieme nei vari metodi che gestiscono l'inizializzazione, la pausa o lo stop di Timer per l'avanzamento della partita. Infine, un altro metodo di notevole importanza è repaintGame() che permette alla schermata di gioco di essere ridisegnata ogni volta che viene chiamato dal metodo startTimer() (ogni 20ms in questo caso).

La classe **View**, che è implementata tramite pattern singleton ed implementa **IView**, funge da facciata, incapsulando la finestra grafica MainGUI, in modo tale da poter rendere il più pulita possibile la comunicazione da e verso il Controller.



Figura 3.3.1: Schermata iniziale dell'applicazione (Menù)



Figura 3.3.2: Schermata di gioco dell'applicazione (partita in corso)

### 3.4 Controller

Le classi appartenenti al blocco Controller di JpVz si trovano nel package `jpvoz.controller`. La loro struttura è rappresentata nel diagramma UML di seguito (figura 3.4)

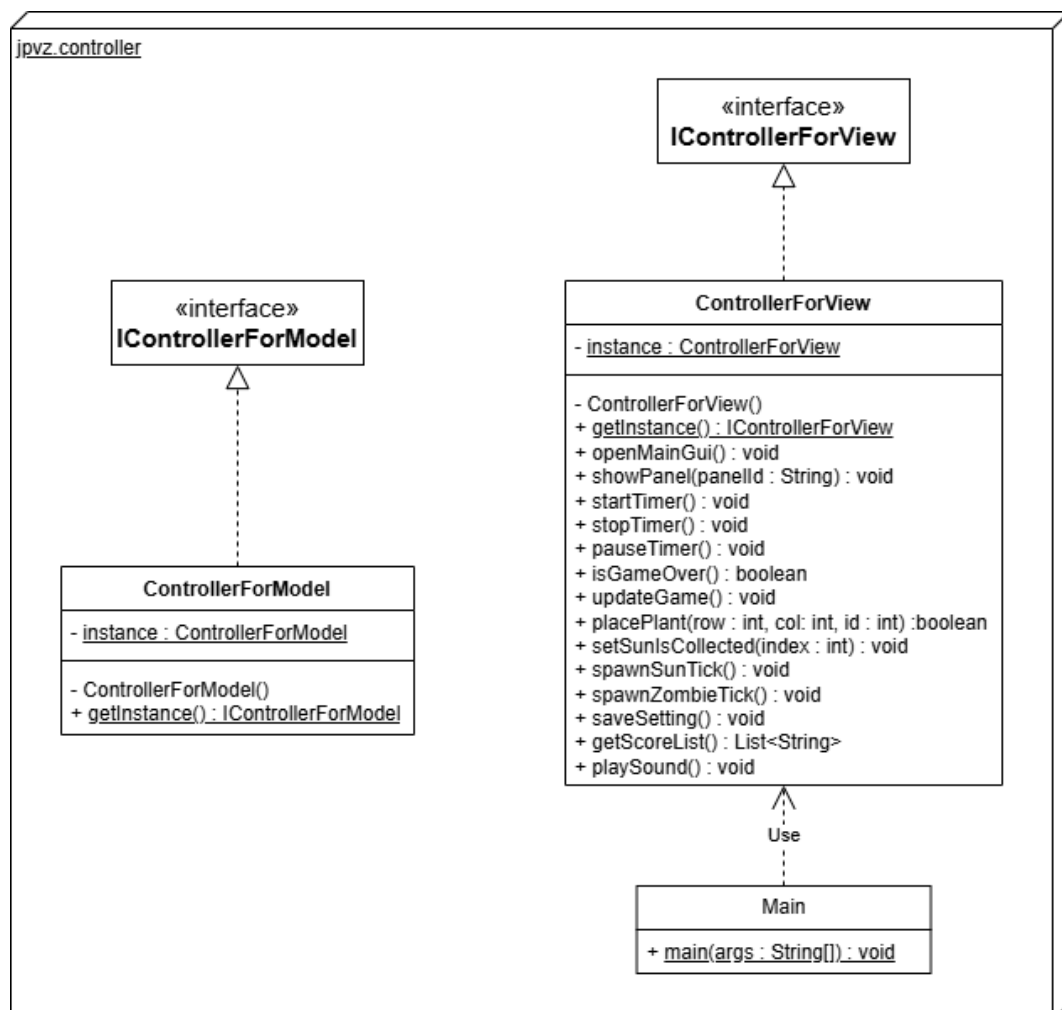


Figura 3.4: Diagramma UML delle classi contenute nel package controller (`jpvoz.controller`)

Il package contiene le classi Main, ControllerForModel e ControllerForView e le rispettive interfacce: IControllerForModel e IControllerForView. In questa architettura software il modulo Controller funge da intermediario tra il Model e la View:

- Coordina il ciclo di simulazione del gioco, sincronizzando tutti i timer
- Fornisce alla View i dati necessari per ridisegnare le schermate adeguatamente in base all'avanzamento di gioco.
- Intercetta gli eventi e gli input dell'utente presi dalla View e li comunica al Model per svolgere le opportune modifiche dello stato logico dell'applicazione.

Entrando nel dettaglio delle varie classi, esse si dividono in tre blocchi principali:

- **Main**: costituisce il punto di ingresso dell'intera applicazione e all'avvio del programma, per avviare la finestra grafica, delega il compito al ControllerForView chiamando il metodo openMainGui().
- **IControllerForModel e ControllerForModel**: quest'ultima classe è implementata tramite pattern singleton e, a sua volta, implementa l'interfaccia IControllerForModel. Rappresentano l'interfaccia dedicata alla comunicazione tra Controller e Model, tuttavia nell'implementazione attuale di JpVZ non ne è stato necessario l'uso, dunque è stata predisposta per preservare la simmetria strutturale del pattern MVC.

**IControllerForView e ControllerForView** svolgono il ruolo più importante del controller, quindi meritano una descrizione più approfondita. La classe ControllerForView è implementata tramite pattern singleton per garantire l'esistenza di un'unica istanza di controllo globale e, a sua volta, implementa IControllerForView per ridurre l'accoppiamento tra le varie classi. Al momento della prima invocazione del metodo statico getInstance(), il costruttore privato si occupa di inizializzare alcuni attributi del Model letti dalla classe Config. Questa classe contiene vari metodi, tra i più importanti troviamo:

- openMainGui(): questo metodo prima carica in memoria tutte le immagini necessarie all'applicazione e inizializza suoni e volumi, in seguito, chiama il metodo della View che si occupa di istanziare la schermata iniziale ovvero MainGUI.
- saveSetting(): metodo che si occupa di salvare i cambiamenti fatti nella schermata delle impostazioni.
- getScoreList(): metodo che si occupa di restituire una lista di stringhe per visualizzare i vari punteggi ottenuti dai giocatori.
- playSound(): metodo che prende una lista di String dal Model contenenti una serie di parole chiave di suoni da riprodurre e, in base a queste, vengono riprodotti dei suoni singolarmente o in loop tramite la classe AudioManager in utils.
- updateGame(): metodo che chiama updateGame() del Model e che, a sua volta, viene richiamato in maniera regolare dal timer di gioco, per aggiornare lo stato della partita.
- Metodi di coordinamento temporale e sincronizzazione degli eventi, che centralizzano il controllo dei vari timer e pilotano la comparsa delle varie entità di gioco come startTimer() e spawnZombieTick().

Oltre ai metodi principali presenti nel diagramma UML, ControllerForView implementa una serie di metodi getter degli attributi di tutte le entità per permettere alla View di disegnare la grafica di gioco, garantendo che questa dipenda solo da attributi primitivi e non dalle classi interne del Model.

### 3.5 Problemi riscontrati

Nonostante la logica dell'applicazione non sia molto complessa, durante lo sviluppo si sono presentate alcune sfide rilevanti:

- **Gestione dei frame e animazioni:** data la mole di immagini si è deciso di creare due classi di supporto (Animation e SpriteManager, entrambe in jpvz.utils) per favorirne il caricamento una sola volta durante il lancio dell'applicazione, per non rischiare cali di prestazione dovuti al ricaricamento in memoria delle immagini ad ogni repaint. Inoltre, è stata una sfida importante anche il sistema di cambio di animazione delle entità. Questo problema è stato risolto aggiungendo un attributo che identifica lo stato dell'entità (ad esempio attacco per le piante o mangia per gli zombie) e con un'alternanza di tick generico o tick specifici a seconda delle animazioni da disegnare, in modo tale da essere fluide e continue.
- **Gestione dei suoni:** la gestione della sovrapposizione di suoni continui o singoli è stata una sfida importante da implementare. Si è deciso di appoggiarsi ad una classe AudioManager in utils che ha metodi di gestione di suoni singoli o in loop, i quali vengono riconosciuti da parole chiave. Per quanto riguarda i suoni singoli le parole chiave vengono inserite in una lista dalla logica del Model mentre per i suoni in loop c'è sempre nel Model una variabile booleana, in seguito il ControllerForView si occupa di riprodurre i vari suoni. Per quanto riguarda invece la colonna sonora dell'applicazione questa viene riprodotta direttamente dal metodo openMainGui del ControllerForView, chiamato all'avvio dal Main, e termina con la chiusura dell'applicazione.
- **Mappatura delle coordinate:** la finestra di gioco consente il ridimensionamento, inoltre l'inizio della griglia di gioco non coincide con l'inizio del GamePanel. È stato quindi necessario usare dei valori percentuali rispetto ai margini della griglia e alle caselle stesse. Grazie a ciò si è ottenuta una buona precisione nel disegno, nei click del mouse e nella rilevazione delle hitbox.

## 4. Possibilità di estensione e personalizzazione

JPvZ è stato progettato tenendo in mente la possibilità di personalizzarne in parte l'aspetto e il funzionamento del gioco. Infatti, è possibile cambiare parametri legati al bilanciamento temporale come la nascita di zombie, dei soli o di velocità di aggiornamento del gameLoop e di cambiare il numero di soli iniziali attraverso il file di config (conf/config.txt) composto da una serie di coppie chiave=valore lette all'avvio dell'applicazione. È anche possibile modificare la mappa, la difficoltà, il volume della musica e il nome del giocatore, sia dal file di configurazione, sia direttamente nel pannello delle impostazioni del gioco.

Inoltre, l'applicazione è anche stata pensata per una scalabilità di nuove possibili entità, infatti, basta aggiungere sottoclassi alle superclassi Plant e Zombie per ottenere nuovi zombie e piante, con rispettive nuove particolarità cambiando attributi o sovrascrivendo i metodi.

L'architettura consente anche di aggiungere nuove ambientazioni dedicate, limitandone il tipo di entità che possono essere presenti, modificando i metodi di nascita o di selezione di esse nel Model.

## 5. Bibliografia

- **Draw.io**, software per la modellazione e realizzazione dei diagrammi UML presenti, <https://app.diagrams.net/>.
- **Plants vs. Zombies Wiki**, per la descrizione e le informazioni legate al gioco originale: "Plants vs. Zombie", [https://plantsvszombies.fandom.com/it/wiki/Plants vs. Zombies](https://plantsvszombies.fandom.com/it/wiki/Plants_vs._Zombies).
- **Plants vs. Zombies Wiki Media Asset**, per il logo in prima pagina, [https://plantsvszombies.fandom.com/it/wiki/Plants vs. Zombies?file=Pvz logo stack ed rgb.png](https://plantsvszombies.fandom.com/it/wiki/Plants_vs._Zombies?file=Pvz_logo_stack_ed_rgb.png).